Data & AI Governance Framework on Databricks Lakehouse

Current architecture · How the structure makes new regulations a config change · UK integration walkthrough

---

The accelerator started as a DPDP-India compliance POC. A recent refactor turned it into a **regulation-adaptive governance platform on the lakehouse**: the core code knows nothing about any specific regulation, and every regulation-specific value lives in a single directory. Adding a new regulation is a configuration change, not a rewrite.

This doc shows what the architecture looks like today, why it's genuinely modular, and walks through a concrete UK-data-protection integration to demonstrate the claim.

Current architecture

Four layers. Layers 1 and 2 are regulation-agnostic code; Layer 3 is regulation- specific configuration; Layer 4 is the user-facing skin.

LAYER 1 — PLATFORM CORE (REGULATION-AGNOSTIC)

Lakehouse + Unity Catalog substrate

Medallion pipeline, column-mask UDFs, row-filter mechanism, audit log, Delta Sharing, consent-event schema, DSR plumbing, Agent Bricks integration, persona orchestrator. Byte-identical across deployments.

▼

LAYER 2 — GOVERNANCE TAXONOMY (SEMI-UNIVERSAL)

Universal PII patterns, rights catalogue, pack loader

11 universal PII patterns (email, phone, card, name, address, DOB, IP, medical, insurance, CVV, bank account), the `PIIPattern` dataclass, the 9-entry `RIGHT_CATALOGUE` superset, and the typed-accessor pack loader.

▼

LAYER 3 — REGULATION PACK (SPECIFIC, SWAPPABLE)

One directory per regulation

Everything regulation-specific: compliance rules, rights activations, notice bodies, retention defaults, residency allow-list, breach SLA, language registry, region-specific PII patterns. Selected by a single environment variable.

▼

LAYER 4 — PERSONA & DELIVERY (REGULATION-AWARE SKIN)

Dashboards, Genie spaces, agent prompts

Four persona roles (CCO / GC / CMO / CFO) constant across regulations. Their dashboard content reads from pack-populated views, so swapping a pack re-skins the dashboards automatically.

Directory map

Where each layer physically lives in the repo. The `regulations/` directory is the extension point: DPDP India sits there today; a UK pack sits as a sibling, same shape; both are loaded simultaneously, with per-row routing by `jurisdiction` column (see ADR-0001).

```
repo root
│
├── governance_core/                  ← Layer 1/2 — regulation-agnostic code
│   ├── pack_loader.py                   loads every pack under regulations/
│   ├── resolver.py                      pack_for(jurisdiction); cross-cutting union
│   ├── rights.py                        9-entry RIGHT_CATALOGUE superset
│   ├── consent_model.py                 consent-event schema contract
│   └── pii_patterns/
│       └── universal.py                 11 universal PIIPatterns
│
├── regulations/                      ← Layer 3 — swap point
│   │
│   ├── dpdp_2023/                    ── active today (India)
│   │   ├── pack.yaml
│   │   ├── rules.yaml
│   │   ├── rights.yaml
│   │   ├── notices.yaml
│   │   ├── retention_defaults.yaml
│   │   ├── residency.yaml
│   │   ├── breach_sla.yaml
│   │   ├── languages.yaml
│   │   └── pii_patterns.py
│   │
│   └── uk_gdpr/                      ── future pack (lands here, same 9 files)
│       ├── pack.yaml                    jurisdiction=GB, ICO, £17.5M ceiling
│       ├── rules.yaml                   UK GDPR articles + DPA 2018 sections
│       ├── rights.yaml                  7 rights activated
│       ├── notices.yaml                 en-GB body
│       ├── retention_defaults.yaml
│       ├── residency.yaml               UK + adequacy list
│       ├── breach_sla.yaml              72h to ICO
│       ├── languages.yaml
│       └── pii_patterns.py              NHS, NINO, UK postcode, UTR
│
├── pipelines/                        ← Layer 1 — no edits needed for new regulations
│   ├── medallion.py                     Bronze → Silver DLT
│   ├── classification_dlt.py            PII classifier (every loaded pack's patterns)
│   ├── phase1_bootstrap.py              reads all packs, populates compliance tables
│   └── retention_enforcement.py
│
├── schemas/                          ← Layer 1 — no edits needed
│   └── pii_patterns.py                  composition shim (universal + every loaded pack)
│
├── scripts/                          ← Layer 1 — no edits needed
│   ├── setup_all_personas.py            persona orchestrator (regulation-agnostic)
│   ├── apply_pii_masks.py
│   ├── apply_persona_row_filters.py
│   ├── dsr_discovery.py
│   ├── dsr_erasure.py                   reads pack for reason / locale / SLA
│   └── generate_multilang_notices.py    reads pack.languages()
│
└── dashboards/                       ← Layer 4 — pack-aware skin
    └── personas/                        CCO / GC / CMO / CFO (constant across regs)
```

Two observations that make the modularity visible:

- **Only `regulations/` grows** when a new regulation is added. The UK pack is 9 files next to the 9 DPDP files — no file under any other directory changes.

- **All packs in `regulations/` are loaded simultaneously.** Each customer-level row carries a `jurisdiction` column that routes rule evaluation to the pack governing that principal. An Indian SaaS with UK clients runs both `dpdp_2023` and `uk_gdpr` active in the same workspace, applying DPDP rules to Indian principals (730-day retention) and UK GDPR rules to UK principals (90-day retention) in the same database. See ADR-0001 for the per-data-subject routing design and 24 enumerated edge cases. (Earlier drafts of this doc described a single-pack-active model selected by a `REGULATION_PACK` env var; that approach was rejected as Mode 2 in ADR-0001's "Alternatives considered" because forcing the strictest rule across packs onto every row destroys the product's economic case for multinational customers.)

What's modular and what's regulation-specific

The modularity claim rests on one invariant: **no regulation-specific value appears as a literal in any file under `pipelines/`, `scripts/`, or `schemas/`**. All such values go through the pack loader.

IN THE CORE (REGULATION-AGNOSTIC)	IN THE PACK (ONE FILE PER CONCERN)
• Medallion pipeline (Bronze → Silver) • DLT classification (universal patterns) • Column masks • Row filters • UC grants (persona layer) • Audit log queries • Delta Sharing • DSR plumbing (discovery + erasure) • Retention enforcement • Persona orchestrator • Universal PII patterns + rights catalogue • Pack loader (typed accessors)	• `pack.yaml` — jurisdiction, authority, penalties • `rules.yaml` — compliance rules + citations • `rights.yaml` — activated rights + SLAs • `notices.yaml` — notice bodies + consent purposes • `retention_defaults.yaml` — per-purpose retention • `residency.yaml` — allowed countries + filter targets • `breach_sla.yaml` — notification deadlines • `languages.yaml` — locale registry • `pii_patterns.py` — region-specific patterns

The pack loader exposes typed accessors — consumers don't read yaml directly:

```
from governance_core.pack_loader import loaded_packs, pack_for

# Multi-pack iteration — every pack under regulations/ at deploy time:
for pack in loaded_packs():
    for rule in pack.rules(): ...

# Per-row routing — pick the pack governing this principal:
pack = pack_for(jurisdiction="IN")
retention_days = pack.retention_default(purpose="marketing_email")  # 730 (DPDP)
pack = pack_for(jurisdiction="GB")
retention_days = pack.retention_default(purpose="marketing_email")  # 90 (UK GDPR)
```

Adding a regulation is a one-touch operation — author a pack directory under `regulations/`, redeploy. Nothing under `pipelines/`, `scripts/`, or `schemas/` changes. The new pack becomes available to any principal whose `jurisdiction` matches its declared `jurisdiction` code.

Integrating the UK data protection regime (walkthrough)

Concrete demonstration. The POC currently ships DPDP-India only; the example below is what a UK pack would look like when that phase begins. It shows the *entire* authoring surface — nothing outside this directory would change.

Directory layout

```
regulations/
├── dpdp_2023/                  ← already shipped (India)
└── uk_gdpr/                    ← authored when the UK phase begins
    ├── pack.yaml               jurisdiction=GB, ICO, £17.5M ceiling
    ├── rules.yaml              rules keyed to DPA 2018 + UK GDPR articles
    ├── rights.yaml             7 rights activated
    ├── notices.yaml            en-GB body, consent legal basis
    ├── retention_defaults.yaml per-purpose retention (storage limitation)
    ├── residency.yaml          UK + adequacy list
    ├── breach_sla.yaml         72h to ICO + high-risk subject notification
    ├── languages.yaml          en-GB primary, cy-GB / gd-GB optional
    └── pii_patterns.py         NHS number, NINO, UK postcode, UTR
```

Representative pack contents

`pack.yaml`:

```yaml
name: "UK Data Protection Act 2018 (as amended by UK GDPR 2021)"
code: uk_gdpr
jurisdiction: GB
supervising_authority: "Information Commissioner's Office (ICO)"
primary_locale: en-GB
max_penalty:
  - { amount: 17_500_000, currency: GBP, trigger: "higher-tier infringement" }
  - { amount:  8_700_000, currency: GBP, trigger: "standard infringement" }
activates:
  rights: [access, rectification, erasure, portability, objection, restriction, no_auto_decision]
  pii_pattern_packs: [universal, uk]
  minor_age: 13
  consent_default: opt_in
```

`pii_patterns.py` (UK-specific):

```
from governance_core.pii_patterns.universal import PIIPattern, ...

NHS_NUMBER = PIIPattern(
    pattern_id="nhs_number", category=CATEGORY_HEALTH, sensitivity=SENSITIVITY_CRITICAL,
    regex_pattern=r"\b\d{3}[\s-]?\d{3}[\s-]?\d{4}\b",
    column_hints=["nhs", "nhs_number", "patient_nhs"], ...)

NINO = PIIPattern(
    pattern_id="uk_nino", category=CATEGORY_DIRECT_GOV, sensitivity=SENSITIVITY_CRITICAL,
    regex_pattern=r"\b[A-CEGHJ-PR-TW-Z]{2}\d{6}[A-D]\b", ...)

UK_POSTCODE = PIIPattern(...)
UTR         = PIIPattern(...)       # Unique Taxpayer Reference

IN_SPECIFIC_PATTERNS = [NHS_NUMBER, NINO, UK_POSTCODE, UTR]
```

The other files (`rules.yaml` citing UK GDPR articles, `rights.yaml` activating 7 rights with SLAs, `residency.yaml` listing the UK + adequacy countries, etc.) follow the same straightforward yaml shape.

Activation

```
# Drop the pack files under regulations/uk_gdpr/, then redeploy.
# All packs in regulations/ load simultaneously. UK principals (rows where
# customers_tagged.jurisdiction = 'GB') automatically pick up UK GDPR rules
# on next phase1_bootstrap run; Indian principals continue under DPDP.
databricks bundle run phase1_bootstrap --target dev
```

What the deploy then produces

TABLE / VIEW	BEFORE (DPDP)	AFTER (UK)
`bronze.compliance_rules`	Rules citing DPDP Sec. 5, 8, 11, 16, 33	Rules citing UK GDPR Art. 5 / 17 / 32 + DPA 2018
`compliance.notice_versions`	3 seeded + 7 machine-translated Indian languages	en-GB primary + any locales declared in `languages.yaml`
`consent_events_log.retention_duration_days`	Per-purpose DPDP values	Per-purpose UK values (storage-limitation)
`compliance.residency_filter` UDF body	`country IN ('India')`	`country IN ('United Kingdom', …)`
`silver.pii_findings`	Includes Aadhaar, PAN, IFSC	Includes NHS Number, NINO, UK Postcode
DSR erasure audit trail	`en-IN` locale + `"DPDP Sec. 12(b) right to erasure"`	`en-GB` locale + `"UK GDPR Art. 17 right to erasure"`

What does NOT change

- Zero edits to `pipelines/` , `scripts/` , or `schemas/` .

- Zero edits to `governance_core/` — universal patterns + pack loader are regulation-agnostic.
- Zero edits to persona dashboards or Genie spaces — they read from pack-populated views.
- Existing integration test suites continue to pass — they assert structural invariants that survive pack swaps.

Effort

The yaml-editing portion is roughly 1–3 working days for a contributor who understands the regulation. The larger exercise is legal review of the notice body and rule citations — a domain-expert task, outside the framework's scope. The framework removes the engineering bottleneck; the legal review remains human work per pack.

> **The claim made concrete:** adding a second regulation is authoring 9 files inside `regulations/uk_gdpr/` — it is not a refactor, not a parallel build, not a fork. The pack directory is the complete authoring surface.

Why Unity Catalog specifically

Every governance primitive the accelerator uses is UC-native. A customer gets a turnkey UC governance posture; the pack is the wrapper for their regulatory context.

GOVERNANCE CONTROL	UC-NATIVE	FRAMEWORK BENEFIT
Column tags	Built-in	PII classification writes directly to UC — lineage, searchability, Catalog Explorer discovery come free
Column masks + row filters	Built-in	Tokenisation + jurisdiction scoping at query time, enforced on every UI path (SQL editor, Genie, REST, notebook, dashboard)
Table / view grants	Built-in	Persona layer is 100% UC grants — no app-tier RBAC to maintain
Audit log (`system.access.audit`)	Built-in	Record-keeping obligations across regulations answered by one query
Lineage	Built-in	DSR discovery walks the lineage graph without customisation
Delta Sharing	Built-in	External auditor access via read-only shares — no accounts to provision
AI Functions + serving endpoints	UC-registered	Classification + multilang notice generation run inside the workspace boundary